

Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales
Departamento de Computación

Liquid Types para la verificación de smart contracts

Tesis de Licenciatura en Ciencias de la Computación

Gonzalo Consoli

Director: Hernán Melgratti

Buenos Aires, 2025

LIQUID TYPES PARA LA VERIFICACIÓN DE SMART CONTRACTS

En este trabajo se desarrolló una herramienta para la verificación de smart contracts escritos en Solidity. La estrategia empleada consiste en traducir el código fuente de Solidity a Haskell, utilizando las instrucciones require y assert como base para generar refinement types, los cuales se interpretan como precondiciones y postcondiciones. Estos tipos refinados se utilizan posteriormente en el proceso de verificación mediante Liquid Haskell.

Además, se presentan ejemplos de uso con contratos de Azure y Rosetta, así como adaptaciones específicas para esta herramienta. Finalmente, se incluye una comparación con otras herramientas de verificación existentes, como SMTChecker y Slither.

Palabras claves: Contratos inteligentes, Solidity, Blockchain, Análisis estático, Tipos refinados, Liquid Haskell, Validación de software

Índice general

1.. Alcance 1
1.1. Contexto 1
1.2. Problema 2
1.3. Contribuciones 2
1.4. Estructura de la tesis 2

2.. Background 5
2.1. Smart Contracts 5
2.2. Liquid Types y Refinement Types 7

3.. Solidity to Haskell 11
3.1. Traducción 11
3.2. Liquidating 13

4.. Validation 15
4.1. Casos de uso 15
4.2. Correctitud 18

5.. Related work 21
5.1. Comparativa con SMTChecker 21
5.2. Slither 22
5.3. Tabla comparativa 25

6.. Conclusión 27
6.1. Trabajo futuro 27

7.. Appendix 31

1. ALCANCE

1.1. Contexto

Los smart contract son parte principal de las DApps, ya que ayudan a automatizar el flujo de trabajo, ejecutar acuerdos sin un intermediario a través de oráculos y manejar activos en blockchains como Ethereum. Pero que un contrato carezca de bugs o errores es algo prioritario en el desarrollo de smart contracts, porque una vez desplegado, el código es inmutable y no es posible la corrección de errores a menos que se despleguen nuevas versiones del contrato. En la práctica, tanto las vulnerabilidades inadvertidas como las intencionadas pueden tener graves repercusiones. Si bien los exploits de alto nivel [10, 7, 8], que resultaron en perdidas de millones o incluso cientos de millones de dólares tienden a captar la atención del publico, los errores mas sutiles y las interacciones lógicas pueden infligir un daño considerable si permanecen sin detectar.

En los últimos años salieron varias herramientas automáticas y semi-automáticas para enfrentar varios aspectos de la seguridad de los smart contracts, tales como fuzzers, analizadores simbólicos, y verificadores integrados tal como el de Solidity SMTChecker [3, 2]. El problema de estas soluciones es que frecuentemente dependen de heuristicas incompletas o necesitan alguna forma de especificación adicional por parte de los desarrolladores lo cual puede aumentar la complejidad de uso. Consecuentemente determinados tipos de errores pueden evitar ser detectados, por lo que se ve la necesidad de metodologías de verificación formal mas robustas que aseguren *safety* y *correctness* [19].

Existen actualmente herramientas para seguridad de smart contracts como Slither, Mythril, y SolType, pero tienen notables limitaciones. Slither realiza análisis estático sobre el código fuente pero tiene complicaciones con inlined assembly, lo cual puede llevar a falsos negativos y falsos positivos debido a la simplicidad de sus heuristicas [9, 5], por lo general esta herramienta falla en identificar vulnerabilidades que dependen de comportamientos dinámicos [12]. Mythril utiliza ejecución simbólica sobre el bytecode de la EVM pero se encuentra con problemas de escalabilidad con contratos complejos, lo cual limita su análisis en profundidad [21] y sus aproximaciones inherentes también resultan en falsos positivos y negativos [22]. Y SolType [1] utiliza tipos refinados para verificar propiedad aritméticas ofreciendo fuertes garantías contra errores de overflow y underflows, pero está limitado a solo operaciones aritméticas.

Dados estos desafíos, este trabajo investiga el potencial de traducir smart contracts de Solidity hacia Haskell agregando las anotaciones de Liquid Haskell. Aprovechando el sofisticado sistema de tipos y chequeos estáticos de Liquid Haskell, nuestro objetivo es descubrir inconsistencias y posibles fallos tempranamente en el ciclo de desarrollo, evitando el riesgo de desplegar un smart contract defectuoso. Una ventaja notable de este enfoque radica en su capacidad para la traducción automática y la validación del código Solidity; por el contrario, la limitación es que se depende explícitamente de las directivas `require` y `assert` en el código fuente para codificar especificaciones. Sin embargo, integrar la traducción automática con un marco de verificación mas expresivo presenta una via prometedora para mejorar la fiabilidad de los smart contracts en la practica.

1.2. Problema

Solidity es el lenguaje mas usado para el desarrollo de smart contracts, principalmente sobre la red de Ethereum. Sin embargo, Solidity carece de avanzadas herramientas de verificación como las que se podrian encontrar en otros ecosistemas como por ejemplo en Java, que cuenta con herramientas tales como PMD [13], SonarQube [17] y SpotBugs [18]. Estas herramientas pueden encontrar posibles errores, code smells y vulnerabilidades de seguridad en tiempo de compilación lo que las hacen útiles para los desarrolladores.

En contraste, el desarrollo en Solidity depende en gran medida de revisiones manuales de código, frameworks de testing y aserciones en tiempo de ejecución, las cuales pueden no ser suficientes para detectar errores lógicos sutiles o casos bordes. Hay varias herramientas de verificación disponibles, y cada día aparecen nuevas, pero tienen al menos una de dos desventajas principales:

- Requieren un esfuerzo significativo por parte de los desarrolladores para ser utilizadas de forma efectiva.
- No ofrecen garantías formales de corrección, ya que no son ni *sound* ni *complete*, o bien dependen de técnicas como IA generativa, requiriendo nuevamente la participación activa de los desarrolladores para verificar y ajustar el uso de estas herramientas.

En particular, Liquid Haskell aprovecha los tipos refinados para verificar estáticamente propiedades de correctitud durante tiempo de compilación. Lo que proponemos es automatizar la traducción de los smart contracts en Solidity hacia Haskell con las anotaciones de Liquid Haskell, ofreciendo una manera practica hacia la verificación formal ligera. A través de este proceso de traducción, surgen las siguientes preguntas de investigación:

> *¿Cómo podemos hacer cumplir automáticamente propiedades de corrección formal en contratos inteligentes escritos en Solidity utilizando el sistema de tipos refinados de Liquid Haskell, basándonos únicamente en características estándar como* `require` *y* `assert`*? ¿Y es este enfoque* ***factible*** *y* ***ventajoso*** *en la práctica, en comparación con las herramientas actuales de verificación?*

1.3. Contribuciones

En esta tesis se desarrolló una herramienta para el análisis de smart contracts que realiza una traducción de Solidity a Haskell y aprovecha los `require` y `assert` para crear tipos refinados para las funciones para de esta manera aprovechar el poder de los tipos refinados con Liquid Haskell, también se proveen ejemplos y comparativas con otras herramientas.

1.4. Estructura de la tesis

En la sección 2.1, se introducen los conceptos fundamentales de contratos inteligentes y del lenguaje Solidity. Se presenta un ejemplo ilustrativo y se explican las directivas `require` y `assert`, las cuales constituyen una parte central del enfoque propuesto.

La sección 2.2 introduce la noción de tipos refinados (refinement types), junto con un ejemplo que ilustra su utilidad para especificar y verificar propiedades en tiempo de compilación.

En la sección 3.1, se detalla el funcionamiento de la herramienta desarrollada, describiendo el proceso de traducción automática de contratos en Solidity a código en Haskell. Se presentan ejemplos concretos para mostrar cómo se realiza el mapeo de estructuras y tipos.

La sección 3.2 continúa ese análisis, explicando cómo se generan las anotaciones de Liquid Haskell a partir de las precondiciones y postcondiciones derivadas del código original.

En la sección 4, se presentan diversos casos de uso que validan la herramienta. Se muestran ejemplos concretos y se discute su correctitud mediante análisis empíricos.

Finalmente, en la sección 5, se realiza una comparación con herramientas existentes como SMTChecker y Slither, utilizando los mismos ejemplos para contrastar las capacidades de detección y verificación de cada enfoque.

2. BACKGROUND

2.1. Smart Contracts

Un *smart contract* es un programa auto-ejecutable almacenado en la *blockchain* que se rige por el principio de «*code-is-law*»: una vez desplegado, su ejecución sigue de forma inmutable la lógica codificada. Por eso mismo, cualquier vulnerabilidad o error puede tener consecuencias irreversibles, lo que hace imprescindible garantizar su corrección desde el inicio.

Solidity es actualmente el estándar de facto para desarrollar *smart contracts* que se ejecutan en la Ethereum Virtual Machine (EVM). Se trata de un lenguaje de alto nivel, orientado a objetos y estáticamente tipado, cuyo diseño —influido por C++, Python y JavaScript— ofrece una sintaxis familiar a desarrolladores de distintos entornos. Permiten construir contratos que regulen transacciones, hagan cumplir acuerdos y automatizar complejos flujos de negocio sobre un *ledger* descentralizado e inmutable, y todo sin la necesidad de intermediarios.

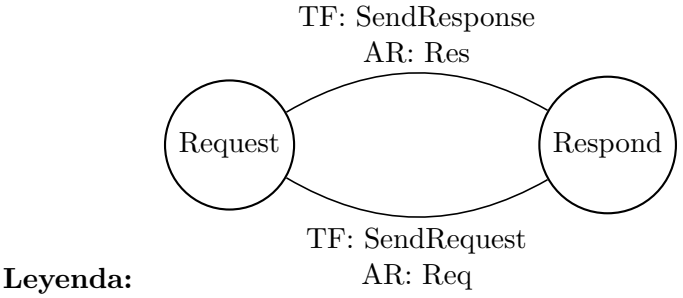
Entre las características clave de Solidity destacan la herencia múltiple con linealización C3, que promueve la modularidad y la reutilización de código; el tipado estático, que obliga a declarar los tipos de variables durante la compilación y permite detectar errores de tipado de forma temprana; un sistema de tipos rico que, además de `bool`, `int` o `address`, incluye `structs`, `enums`, `mappings` y arreglos dinámicos o estáticos; y los mecanismos `require`, `assert` y `revert`, que detienen la ejecución y revierten cambios cuando se violan precondiciones, invariantes internas o se alcanza un estado inválido.

El siguiente contrato, tomado del conjunto Azure Blockchain Workbench [4] y ligeramente modificado para sustituir un `if ... revert` por un `require`, servirá de hilo conductor para ilustrar estas ideas.

```
1  contract HelloBlockchain{
2      enum StateType { Request, Respond}
3      StateType public  State;
4      address public  Requestor;
5      address public  Responder;
6      string public RequestMessage;
7      string public ResponseMessage;
8
9      constructor(string memory message) public{
10         Requestor = msg.sender;
11         RequestMessage = message;
12         State = StateType.Request;
13     }
14
15     function SendRequest(string memory requestMessage) public{
16         require(Requestor == msg.sender);
17         RequestMessage = requestMessage;
18         State = StateType.Request;
19     }
20
21     function SendResponse(string memory responseMessage) public{
22         Responder = msg.sender;
23         ResponseMessage = responseMessage;
```

```
24          State = StateType.Respond;
25      }
26  }
```

Listing 2.1: Contrato `HelloBlockchain`



Leyenda:
TF: Transition Function
AR: Allowed Role
Roles de la aplicación:
Requestor (`Req`)
Responder (`Res`)

Fig. 2.1: Representación gráfica del flujo de estados y roles del contrato `HelloBlockchain`.

La Figura 2.1 muestra los dos estados (`Request` y `Respond`) que maneja el contrato, junto con las funciones de transición y los roles permitidos. Al crearse la instancia, el constructor registra la dirección del `requestor`, almacena el mensaje inicial y fija el estado en `Request`. La función `SendRequest` permite al `requestor` actualizar la solicitud, siempre que se cumpla la precondición de que el remitente coincida con la dirección original, validada mediante `require`. Por su parte, `SendResponse` habilita a otro usuario—el `responder`—a enviar una respuesta y transicionar el contrato al estado `Respond`.

Solidity incorpora herramientas específicas para el manejo de errores y la validación de condiciones. Entre ellas, `require` permite verificar precondiciones externas (como entradas o permisos), revirtiendo la transacción si no se cumplen. Esto se ve en la línea 16 de Listing 2.1, donde se restringe el uso de `SendRequest`. Para comprobar invariantes internas—es decir, condiciones que siempre deben mantenerse—se utiliza `assert`. Cuando se necesita abortar la ejecución bajo condiciones más complejas o personalizadas, puede invocarse `revert`, que además permite devolver un mensaje de error. Estas instrucciones no solo protegen la lógica del contrato, sino que también aseguran transparencia ante errores y estados inválidos.

A pesar de estos mecanismos de seguridad incorporados, garantizar que un *smart contract* sea coherente y correcto en todos sus posibles caminos de ejecución requiere una validación lógica más rigurosa. En este sentido, herramientas como el SMTChecker de Solidity aplican métodos formales para detectar errores potenciales o fallos de seguridad durante el proceso de compilación. Al incorporar este tipo de verificación automática, los desarrolladores pueden reforzar la fiabilidad de sus contratos y reducir los riesgos derivados de vulnerabilidades inadvertidas o errores lógicos sutiles.

Por ejemplo, si después de la linea 18, en la función SendRequest, agregamos un `assert(State == StateType.Respond)`, lo cual obviamente es algo contradictorio, ya

que en la linea 18 definimos `State = StateType.Request` podemos ver como SMTChecker detecta este error a continuación.

```
Warning: CHC: Assertion violation happens here.
Counterexample:
State = 0, Requestor = 0x0, Responder = 0x0, ResponseMessage = []

Transaction trace:
HelloBlockchain.constructor(message){ msg.sender: 0x0 } -- counterexample in-
complete; parameter name used instead of value
State: State = 0, Requestor = 0x0, Responder = 0x0, ResponseMessage = []
HelloBlockchain.SendRequest(requestMessage){ msg.sender: 0x0 } -- counterex-
ample incomplete; parameter name used instead of value
  --> hello.sol:29:9:
   |
29 |         assert(State == StateType.Respond);
   |         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Podemos notar que nos está diciendo que ni siquiera tuvo que instanciar los parámetros para encontrar un contraejemplo, lo que tiene sentido ya que el `assert` es directamente contradictorio.

2.2. Liquid Types y Refinement Types

Los tipos refinados extienden los sistemas de tipos tradicionales incorporando predicados lógicos dentro de las definiciones de tipo [20]. Esto permite un control más preciso sobre los valores que pueden tomar las variables. Por ejemplo, es posible definir un tipo refinado como `NonZero = {v: Int | v != 0}` para garantizar que una variable siempre contenga un número entero distinto de cero.

Liquid Haskell es una extensión del lenguaje Haskell que integra el concepto de tipos refinados. Esta integración permite especificar y verificar propiedades adicionales de correctitud en tiempo de compilación. En particular, los desarrolladores pueden:

- Definir precondiciones precisas para las funciones, asegurando que sean invocadas con argumentos válidos.
- Especificar y mantener invariantes sobre estructuras de datos, garantizando su integridad durante la ejecución.
- Verificar estáticamente propiedades del programa (incluyendo postcondiciones), detectando posibles errores antes de tiempo de ejecución.

Internamente, Liquid Haskell se basa en un SMT solver [20] (como Z3) para verificar automáticamente si se satisfacen los predicados de los tipos refinados. Cuando el compilador encuentra un tipo refinado o una especificación (ya sea precondición o postcondición), traduce dichas restricciones en fórmulas lógicas que son evaluadas por el solver. Si se detecta alguna inconsistencia, el código no compilará, evitando así errores en tiempo de ejecución.

Ejemplo: División segura con denominadores distintos de cero [14]. En Liquid Haskell, la función `die` se utiliza comúnmente para representar código que no debería ejecutarse bajo ninguna circunstancia. Se le asigna un tipo refinado que impone una condición lógicamente falsa, asegurando que solo pueda compilar si el sistema puede demostrar que nunca será llamada:

```
{-@ die :: {v:String | false} -> a @-}
die msg = error msg
```

Este tipo indica que `die` solo puede recibir como argumento una cadena que satisfaga el predicado `false`, lo cual es imposible. Por lo tanto, cualquier uso de `die` obliga a Liquid Haskell a verificar que dicha llamada nunca será alcanzada en tiempo de ejecución.

A continuación, se presenta un ejemplo adaptado desde la documentación oficial de Liquid Haskell para ilustrar cómo este mecanismo puede prevenir errores comunes como la división por cero.

Consideremos la siguiente función en Haskell, llamada `divide'`:

```
divide' :: Int -> Int -> Int
divide' n 0 = die "divide by zero"
divide' n d = n div d
```

A primera vista, parece que la operación `n div d` solo se ejecuta cuando el denominador `d` es distinto de cero. Sin embargo, Liquid Haskell genera una advertencia en la llamada a `die`, indicando que es posible invocar la función `divide'` con un segundo argumento igual a cero.

Este problema puede resolverse asignando a la función un tipo refinado que exija un denominador no nulo:

```
{-@ divide :: Int -> NonZero -> Int @-}
divide _ 0 = die "divide by zero"
divide n d = n div d
```

En este caso, `NonZero` representa un tipo refinado que excluye el valor cero como argumento válido. Al compilar, Liquid Haskell detecta que la rama `divide _ 0` contradice el tipo especificado y la clasifica como código muerto. En otras palabras, garantiza estáticamente que `die "divide by zero"` nunca será ejecutado en tiempo de ejecución.

Como resultado, cualquier uso de la función `divide` debe proporcionar un segundo argumento que se pueda demostrar, en tiempo de compilación, que no es cero. Por ejemplo:

```
avg2 x y = divide (x + y) 2
avg3 x y z = divide (x + y + z) 3
```

Ambos casos son aceptados por Liquid Haskell, ya que `2` y `3` son constantes obviamente distintas de cero. Sin embargo, si intentamos utilizar un valor variable sin una verificación explícita, el sistema lo rechazará. Por ejemplo:

```
f :: Int -> Int -> Int -> Int
f x y z = divide (x + y) z
```

Este código produce un error de tipos como el siguiente:

```
Type Mismatch
.
The inferred type
  VV : {v : GHC.Types.Int | v == z##aJt}
.
```

```
is not a subtype of the required type
  VV : {VV##1090 : GHC.Types.Int | VV##1090 /= 0}
.
in the context
  z##aJt : GHC.Types.Int
Constraint id 1
```

Este mensaje indica que el compilador no puede comprobar estáticamente que `z` sea distinto de cero, por lo que rechaza la llamada a `divide`. Este tipo de verificación estricta es precisamente lo que permite evitar errores en tiempo de ejecución.

En consecuencia, Liquid Haskell constituye un marco robusto para el desarrollo de software seguro y confiable, especialmente en contextos donde la corrección formal es crítica. Su combinación de tipos refinados y verificación automática mediante SMT solvers (como Z3) permite expresar especificaciones detalladas y detectar errores de forma temprana en el proceso de desarrollo.

3. SOLIDITY TO HASKELL

En este capitulo vamos a ver como traducimos los contratos de Solidity a Haskell, y también veremos como inferir las especificaciones de Liquid Haskell.

Eligiendo un lenguaje destino como Haskell podremos aprovechar que ya tiene un framework de liquid types como Liquid Haskell y mapeando instrucciones de Solidity a Haskell podemos aprovechar el pipeline de compilación junto con las herramientas de análisis estático que posee.Veremos que los únicos requerimientos son utilizar las directivas `require` y `assert` que serán utilizadas para las especificaciones de Liquid Haskell.

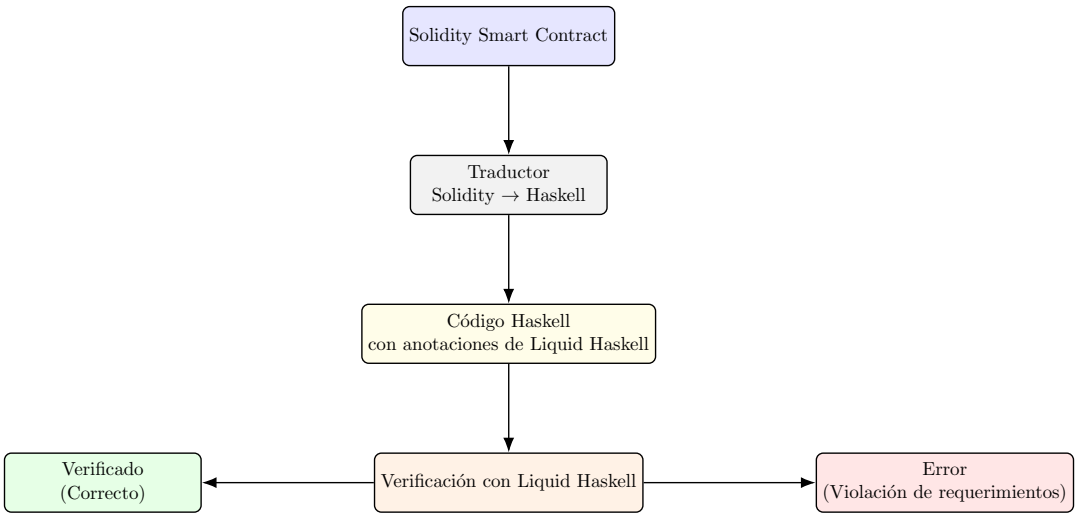


Fig. 3.1: Pipeline de un contrato Solidity hasta la verificación con Liquid Haskell.

3.1. Traducción

El proceso de traducción comienza con el parseo del código fuente de Solidity, el cual es transformado en un árbol de sintaxis abstracta (AST) mediante un parser específico [16]. El AST resultante captura la estructura del contrato inteligente, incluyendo:

- Las variables de estado.
- Las funciones definidas y sus parámetros.
- Las instrucciones de control de flujo.

A partir de esta estructura, la traducción se realiza en varias etapas:

1. **Mapeo de variables de estado:** Las variables de estado del contrato se modelan en Haskell como parte de un Tipo Abstracto de Datos (TAD). Cada una se traduce a su tipo correspondiente en Haskell, según la tabla de equivalencias mostrada en la Figura 3.1.

   Por ejemplo, el siguiente fragmento en Solidity:

Tipo en Solidity	Tipo en Haskell
`uint256`, `uint8`, etc.	`Uint`
`int256`, `int8`, etc.	`Integer`
`bool`	`Bool`
`string`	`String`
`address`	`Address`
`enum MyEnum`	`MyEnum<contractName>`
`struct MyStruct`	`MyStruct<contractName>`
Otros tipos	`UnknownType`

Tab. 3.1: Mapeo de tipos Solidity a Haskell

```
enum StateType { Request, Respond}
StateType public  State;
address public  Requestor;
address public  Responder;
string public RequestMessage;
string public ResponseMessage;
```

Se traduce en Haskell como:

```

data StateTypeHelloBlockchain = RequestHelloBlockchain |
                                RespondHelloBlockchain
                                deriving (Show, Eq)

data HelloBlockchainState = HelloBlockchainState {
  msgHelloBlockchain :: Message,
  blockHelloBlockchain :: Block,
  balanceHelloBlockchain :: Amount,
  stateHelloBlockchain :: StateTypeHelloBlockchain,
  requestorHelloBlockchain :: Address,
  responderHelloBlockchain :: Address,
  requestMessageHelloBlockchain :: String,
  responseMessageHelloBlockchain :: String
}
```

Hacemos notar que las operaciones sobre los tipos básicos como algunos de los mostrados en 3.1 se mapean a la misma operación en Haskell, solo con diferencias sintácticas como por ejemplo el operador `=/` en Haskell y `!=` en Solidity.

Actualmente, la herramienta no ofrece soporte para estructuras de datos como `arrays` o `mappings`, ni para ciertos elementos de control de flujo como bucles (`for`, `while`). Sin embargo, durante el desarrollo se exploró una estrategia experimental para manejar `mappings` de forma manual, aunque esta funcionalidad quedó fuera del alcance de la automatización final debido a restricciones de tiempo y complejidad.

La estrategia consistía en implementar una estructura tipo `Map` en Haskell que permitiera representar los `mappings` de Solidity. Posteriormente, se definía una correspondencia entre las operaciones típicas de `mappings` (como `get` y `set`) y sus equivalentes en dicha estructura. Para completar la integración con Liquid Haskell, también sería necesario definir los predicados correspondientes que permitan expresar propiedades sobre el contenido del mapa en las anotaciones de tipos refinados.

2. **Traducción de funciones:** Cada función de Solidity es traducida en una función de Haskell. Los parámetros de la función y el tipo de retorno son explícitamente definidos en Solidity, por lo que se utilizaran los mismos para la traducción.Se soportan dos tipos de funciones de *las cuales las podemos diferenciar mirando* subNode.stateMutability que nos indica como la función interactúa con la blockchain:

   - **Funciones Pure/View:** Estas funciones no modifican el estado del contrato y son traducidas en funciones de Haskell que computan un valor sin alterar el estado general del contrato.
   - **Funciones que modifican el estado:** Estas funciones alteran el estado del contrato y son traducidas en funciones de Haskell que retornan un estado actualizado del contrato.

   Por ejemplo, la función SendRequest de nuestro ejemplo:

```
1 sendRequest st requestMessage = let
2   st1 = if ((requestorHelloBlockchain st)==(sender (
      msgHelloBlockchain st)))
3         then st else revert "error"
4   st2 = st1 { requestMessageHelloBlockchain = requestMessage }
5   st3 = st2 { stateHelloBlockchain = RequestHelloBlockchain }
6     in
7       st3
```

3. **Instrucciones de control de flujo:**

   - **if:** Son convertidos en su expresión equivalente en Haskell.
   - **require:** Son convertidos en un if asegurando la condicion dada y retorna un error en caso contrario, también veremos mas adelante que son usados para las precondiciones de Liquid Haskell.
   - **assert:** Lo salteamos como instrucción de Haskell en si pero veremos que lo usaremos para las postcondiciones de Liquid Haskell.
   - La ejecución secuencial de Solidity es modelada en Haskell usando la expresión `let` y encadenando actualizaciones al estado del contrato.

3.2. Liquidating

Denominamos etapa de *liquidating* al proceso de enriquecer el código generado en Haskell con anotaciones propias de Liquid Haskell. Este paso nos permite especificar precondiciones y postcondiciones para las funciones, facilitando la verificación formal de propiedades sobre el estado y los argumentos del contrato.

A continuación se detallan las dos clases principales de anotaciones que se generan:

Precondiciones: Estas se derivan directamente de las instrucciones `require` presentes en el código original de Solidity. Su propósito es garantizar que las funciones se invoquen con argumentos válidos y bajo condiciones permitidas.

El procedimiento comienza extrayendo todas las sentencias `require` del cuerpo de la función objetivo. Luego, cada una se traduce en un predicado válido dentro del sistema de tipos refinados de Liquid Haskell. Este proceso considera si la condición hace referencia a variables de estado del contrato o a parámetros de entrada de la función. Además, la

expresión dentro del `require` se traduce de forma recursiva, contemplando operaciones binarias u otras construcciones si fuera necesario.

Una vez traducidas todas las condiciones, se genera la parte correspondiente a las precondiciones en la anotación de tipo de la función. Esta sección antecede a las posibles postcondiciones.

Por ejemplo, si analizamos la función `SendRequest`, que contiene un único `require`, obtendremos la siguiente anotación en Liquid Haskell:

```
{-@ sendRequest ::{
                    st:HelloBlockchainState |(requestorHelloBlockchain st)
                        == (sender (msgHelloBlockchain st)))
                }
                    -> {requestMessage:String | True}
                    -> {v:HelloBlockchainState | True}
@-}
```

Postcondiciones: Estas se derivan de las instrucciones `assert` y permiten expresar propiedades que deben cumplirse al finalizar la ejecución de la función, ya sea sobre el valor de retorno o sobre el nuevo estado del contrato.

El proceso de traducción es similar al de las precondiciones: se recolectan todas las expresiones `assert`, se interpretan en función de si afectan al estado o al valor retornado, y se integran al tipo refinado como parte de la especificación de salida.

En el ejemplo anterior, la función `SendRequest` originalmente no incluye ninguna postcondición explícita. No obstante, podemos extenderla con dos `assert` ilustrativos que validen el nuevo estado del contrato:

```
function SendRequest(string memory requestMessage) public {
require(Requestor == msg.sender);
RequestMessage = requestMessage;
State = StateType.Request;
assert(RequestMessage == requestMessage);
assert(State == StateType.Request);
}
```

Esto produce una nueva anotación refinada que refleja las garantías introducidas por los `assert`:

```
{-@ sendRequest :: {st:HelloBlockchainState |
                        ((requestorHelloBlockchain st) ==
                        (sender (msgHelloBlockchain st)))
                    }
                -> {requestMessage:String | True}
                -> {v:HelloBlockchainState |
                    ((requestMessageHelloBlockchain v) == requestMessage)
                        &&
                    ((stateHelloBlockchain v) == RequestHelloBlockchain)
                    }
@-}
```

Este enriquecimiento del código permite que Liquid Haskell verifique automáticamente que, bajo las condiciones requeridas, la función siempre cumple las propiedades especificadas al finalizar su ejecución.

4. VALIDATION

En este capítulo evaluamos el comportamiento de la herramienta propuesta aplicándola a una serie de contratos inteligentes diseñados para ilustrar casos relevantes de verificación. El objetivo es analizar, mediante ejemplos concretos, si la traducción automática desde Solidity hacia Haskell con anotaciones de Liquid Haskell permite detectar fallos lógicos, violaciones de precondiciones y condiciones inalcanzables que no siempre son capturados por otras herramientas.

Para ello, se presentan casos de uso progresivos, que van desde ejemplos simples diseñados para validar el funcionamiento básico del sistema, hasta escenarios más elaborados tomados de contratos reales o adaptados desde workbenchs como Rosetta y Azure. En cada caso, mostramos tanto el código original en Solidity como su correspondiente traducción a Haskell, acompañados del análisis generado por nuestra herramienta.

4.1. Casos de uso

Ejemplo Simple

Para comenzar con la discusión, presentaremos un ejemplo concreto. Empezaremos reforzando las postcondiciones de la función `SendRequest`, tal como está definida en el smart contract referenciado en Listing 2.1. Este enfoque permite que nuestra herramienta reconozca las sentencias `assert` como postcondiciones y, posteriormente, valide su corrección.

```
1 function SendRequest(string memory requestMessage) public{
2     require(Requestor == msg.sender);
3     RequestMessage = requestMessage;
4     State = StateType.Request;
5     assert(State == StateType.Request);
6     assert(RequestMessage == requestMessage);
7     }
```

Ahora, supongamos que cometemos un error al realizar las asignaciones:

```
1 function SendRequest(string memory requestMessage) public{
2     require(Requestor == msg.sender);
3     RequestMessage = requestMessage;
4     State = StateType.Respond;   //introducimos error
5     assert(State == StateType.Request);
6     assert(RequestMessage == requestMessage);
7     }
```

Nuestra herramienta es capaz de detectar el error en este ejemplo.

Ejemplo Bazaar

El Listing 4.1 muestra un contrato en Solidity simplificado, compuesto por dos partes. El primer contrato, `Bazaar`, incluye una función para verificar si un precio es válido; el segundo contrato, `ItemListing`, demuestra cómo se utiliza esta verificación en tres escenarios distintos, dos de los cuales revierten bajo ciertas condiciones.

```
pragma solidity ^0.8.0;

contract Bazaar {
    address public InstanceBazaarMaintainer;

    constructor() {
        InstanceBazaarMaintainer = msg.sender;
    }

    function ValidPrice(int itemPrice) public pure returns (bool) {
        require(itemPrice > 0, "The price must be greater than 0");
        bool result = true;
        assert(result == true);
        return result;
    }
}

contract ItemListing {
    bool flag = true;

    function BuyItemRevert_WrongCall() public {
        Bazaar bazaar = new Bazaar();
        flag = bazaar.ValidPrice(-5);
    }

    function BuyItemNoRevert() public {
        Bazaar bazaar = new Bazaar();
        if (!bazaar.ValidPrice(50)) {
            revert("It is not a valid price");
        }
        flag = true;
        assert(flag == true);
    }

    function BuyItemRevert_ReturnsTrue() public {
        Bazaar bazaar = new Bazaar();
        if (bazaar.ValidPrice(50)) {
            revert("It is not a valid price");
        }
        flag = true;
        assert(flag == true);
    }
}
```

Listing 4.1: Examples of revert

Los siguientes fragmentos de código muestran cómo nuestra herramienta traduce y analiza dos de las funciones anteriores, identificando escenarios en los que una ejecución con `revert` es inevitable.

```
{-@ buyItemRevert_WrongCall :: {st:ItemListingState | True} ->
                               {v:ItemListingState | True}
@-}
buyItemRevert_WrongCall :: ItemListingState -> ItemListingState
buyItemRevert_WrongCall st =
  let bazaar = constructorBazaar (msgItemListing st) (blockItemListing st)
      st1    = st { flagItemListing = validPrice bazaar -5 }
```

```
8   in st1
```

Listing 4.2: Error in `buyItemRevert_WrongCall`

En el listado 4.2, la función `buyItemRevert_WrongCall` intenta llamar a `validPrice` con un precio negativo (`-5`), lo cual viola la precondición que exige que el precio del ítem sea mayor a cero. Nuestra herramienta detecta esta infracción y marca la función como una que inevitablemente ejecutará un `revert`, en lugar de finalizar correctamente.

```
1  {-@ buyItemRevert_ReturnsTrue :: {st:ItemListingState | True} ->
2          {v:ItemListingState | (((flagItemListing v)) == (True))}
3   @-}
4  buyItemRevert_ReturnsTrue :: ItemListingState -> ItemListingState
5  buyItemRevert_ReturnsTrue st =
6    let bazaar = constructorBazaar (msgItemListing st) (blockItemListing st)
7        st1    = if validPrice bazaar 50
8                   then revert "It is not a valid price"
9                   else st
10       st2    = st1 { flagItemListing = True }
11       st3    = st2 -- assert statement
12   in st3
```

Listing 4.3: Error in `buyItemRevert_ReturnsTrue`

En el listado 4.3, la función `buyItemRevert_ReturnsTrue` ejecuta un `revert` cuando la llamada a `validPrice bazaar 50` evalúa como `True`. A pesar de que la postcondición establece que `flagItemListing` debe ser igual a `True`, la ejecución de `revert('It is not a valid price')` impide que se alcance ese estado final. Como resultado, el analizador informa una ejecución que siempre finalizará con `revert`, evidenciando una discrepancia lógica entre la postcondición declarada y el comportamiento real del código.

Ejemplo Bet

El siguiente código está adaptado del contrato `Bet` del crestomatía Rosetta [6]. Nuestra herramienta confirma que el contrato es seguro en su forma original. Sin embargo, al introducir una tercera transferencia dentro de la función `timeout`, la herramienta detecta un error inevitable que supera el balance total disponible.

```
1  function timeout_wrong() external payable {
2      require(deadline < block.number, "The bets are still open");
3      require(address(this).balance == 2*wager, "Invalid Balance");
4      (bool success,) = player1.call{value: wager}("");
5      require(success, "Transfer failed.");
6      (bool success2,) = player2.call{value: wager}("");
7      require(success2, "Transfer failed.");
8      (bool success3,) = player2.call{value: wager}("");
9      require(success3, "Transfer failed.");
10 }
```

En este caso, el contrato garantiza que haya exactamente `2 * wager` fondos disponibles para cubrir dos transferencias. Sin embargo, al intentar realizar una tercera transferencia, se intenta consumir más Ether del que el contrato posee, lo cual provoca que la última transferencia falle y que la ejecución termine con un `revert`.

```
1  {-@ timeout_wrong :: {st:BetState | (((deadlineBet st)) <
2                                       (number (blockBet st)))
3                                  && ((balanceBet st) == (2 * (wagerBet st)))}
```

```
4                   -> {v:BetState | True}
5  @-}
6  timeout_wrong :: BetState -> BetState
7  timeout_wrong st = let
8    st1 = if (deadlineBet st < number (blockBet st))
9          then st else revert "error"
10   st2 = if (balanceBet st1 == 2 * wagerBet st1)
11         then st1 else revert "error"
12   st3 = callBet st2 (player1Bet st2) (wagerBet st2)
13   st4 = st3 -- skipped require statement
14   st5 = callBet st4 (player2Bet st4) (wagerBet st4)
15   st6 = st5 -- skipped require statement
16   st7 = callBet st6 (player2Bet st6) (wagerBet st6)    -- Third transfer
17   st8 = st7 -- skipped require statement
18 in
19   st8
```

Listing 4.4: Error in `timeout_wrong`

En Listing 4.4, la función `timeout_wrong` comienza aplicando dos precondiciones mediante sentencias `if`: el número de bloque actual debe superar `deadlineBet`, y el balance debe ser igual a `2 * wagerBet`. Esto está alineado con el requisito en Solidity de que el contrato solo debe contar con Ether suficiente para dos pagos.

Luego de estas validaciones, el código intenta realizar tres transferencias (representadas por `callBet`). La tercera transferencia inevitablemente falla, ya que el contrato solo dispone de fondos suficientes para las dos primeras. Como resultado, nuestro analizador marca la función `timeout_wrong` como una con ejecución garantizada de `revert`, lo que refleja una contradicción lógica entre el balance verificado y la cantidad de pagos que se intenta realizar.

4.2. Correctitud

En este trabajo no se realizó una prueba formal de correctitud, ya que dicha tarea excede el alcance definido para esta tesis. No obstante, se adoptó una estrategia práctica para respaldar la validez del enfoque propuesto.

Se utilizó un conjunto de contratos inteligentes escritos en Solidity como benchmark, abarcando diversas características del lenguaje actualmente soportadas por la herramienta. El objetivo fue verificar que, dado un contrato válido en Solidity (compuesto únicamente por construcciones compatibles), la herramienta genere código Haskell que compile sin errores.

El código traducido se valida utilizando el compilador Glasgow Haskell Compiler (GHC) [11] y luego Liquid Haskell, lo cual garantiza que los tipos refinados generados estén bien formados y que la traducción no introduzca errores de tipado. Adicionalmente, se llevó a cabo una inspección manual del código Haskell resultante para corroborar que el comportamiento observable tras la traducción sea coherente con la semántica esperada del contrato original en Solidity. Esta revisión incluyó la correcta preservación de condiciones expresadas mediante `require` y `assert`.

Los resultados obtenidos muestran que los contratos válidos, que respetan todas las condiciones impuestas, generan código Haskell igualmente válido y correctamente tipado. En cambio, los contratos que contienen condiciones que se violan—por ejemplo, fallos en `require` o `assert`—producen código que no supera la verificación de Liquid Haskell.

Si bien este enfoque no constituye una demostración formal de correctitud, sí ofrece evidencia empírica suficiente para afirmar que la herramienta preserva el comportamiento deseado y permite detectar violaciones a propiedades especificadas, lo cual refuerza su utilidad práctica en contextos de desarrollo seguro de contratos inteligentes.

En la siguiente tabla se presentan los contratos del conjunto de ejemplos provistos por Azure, junto con una evaluación sobre su traducibilidad utilizando nuestra herramienta. Para cada contrato, se indica si fue traducido correctamente y, en caso contrario, se explicita la razón principal por la cual no fue posible completar el proceso.

Contract Name	**Reason for Not Being Translatable**
DigitalLocker	OK
AssetTransfer	OK
BasicProvenance	OK
HelloBlockchain	OK
RefrigeratedTransportation	OK
RoomThermostat	OK
SimpleMarketplace	OK
DefectiveComponentCounter	Contains an array
FrequentFlyerRewardsCalculator	Contains an array
PingPongGame.sol	Contains an event
Baazar	Calls a function that is neither view nor pure

Tab. 4.1: Contracts and Reasons for Non-Translatability

De los 11 contratos analizados, 7 fueron traducidos exitosamente sin requerir modificaciones. Esto representa aproximadamente el 63 % de cobertura efectiva sobre el conjunto de Azure. Los contratos no traducibles actualmente dependen de características del lenguaje que aún no han sido implementadas en nuestra herramienta, como arreglos, eventos o restricciones sobre funciones externas no puras.

Este resultado es consistente con el enfoque incremental de desarrollo adoptado en este trabajo, y sugiere que la extensión del sistema para cubrir dichos elementos permitiría aumentar significativamente la cobertura práctica del traductor.

5. RELATED WORK

5.1. Comparativa con SMTChecker

La herramienta integrada en el compilador de Solidity, SMTChecker [15], utiliza *Satisfiability Modulo Theories* (SMT) para realizar análisis estático sobre contratos inteligentes. Es particularmente eficaz en la detección de errores comunes como underflows y overflows aritméticos, divisiones por cero, condiciones triviales, segmentos de código inalcanzables, accesos fuera de rango en arreglos, y transferencias sin saldo suficiente.

Ejemplo Simple

Tanto nuestra herramienta como SMTChecker detectan correctamente el error presente en este ejemplo.

Ejemplo Bazaar

En estos ejemplos, nuestra herramienta identifica correctamente que dos funciones revertirán de forma inevitable. En particular, una de ellas intenta pasar un precio negativo a una función que contiene una instrucción `require` que exige un valor positivo; la otra función ejecuta un `revert` cada vez que se proporciona un precio válido. Aunque estos comportamientos son evidentes mediante una inspección manual, el verificador SMTChecker de Solidity no siempre marca estas rutas como errores. Consideramos que esta discrepancia se debe a las diferencias en cómo cada herramienta trata las llamadas externas y evalúa las condiciones de reversión:

Primero, **las llamadas externas son tratadas como desconocidas por defecto.** El SMTChecker modela típicamente cualquier llamada a una función externa como una invocación arbitraria o potencialmente maliciosa. Incluso cuando el código fuente del contrato externo está disponible en tiempo de compilación, su modelo conservador asume que el contrato desplegado podría no coincidir con el analizado. Como resultado, no siempre puede concluir que una llamada externa revertirá, a menos que exista una contradicción directa (por ejemplo, una instrucción `assert(false)`).

En segundo lugar, está el manejo de **instanciaciones con `new`**. En nuestro código, creamos explícitamente una nueva instancia del contrato externo en cada llamada. Esto permite que el SMTChecker "vea"la implementación concreta. No obstante, si la ejecución de un `revert` no viola explícitamente una propiedad interna o una aserción definida por el usuario, el SMTChecker puede interpretar esa ruta de ejecución como válida y no reportarla como un error.

Por último, se encuentra la diferencia entre **propiedades por defecto y propiedades definidas por el usuario.** El SMTChecker se enfoca principalmente en propiedades predefinidas, como comprobaciones de overflows de enteros, código inalcanzable o aserciones explícitas no manejadas. No necesariamente marcará una función que "siempre revierte" como problemática, a menos que dicho comportamiento viole alguna propiedad activa en el análisis. En contraste, nuestra herramienta reporta cualquier ruta de ejecución que conduzca incondicionalmente a una instrucción `revert`, utilizando una técnica de exploración de caminos más explícita.

Ejemplo Bet

El comportamiento de la función `timeout_wrong()` revela una limitación importante del análisis realizado por el SMTChecker, relacionada con la forma en que modela el balance del contrato durante la ejecución. Esta función intenta realizar tres transferencias de Ether por un valor igual a `wager`, a pesar de que el contrato solo posee un total de `2 * wager`. Como resultado, la tercera transferencia fallará inevitablemente y provocará una ejecución con `revert`.

Sin embargo, esta situación no es marcada como un error por el SMTChecker, por varios motivos:

Primero, como ya se discutió en la sección 5.1, el SMTChecker no considera por defecto que un `revert` sea un fallo, a menos que viole una propiedad explícita o una aserción definida por el desarrollador.

Segundo, el análisis del balance del contrato se realiza de manera conservadora. El SMTChecker no asume que el balance se mantiene fijo entre llamadas y permite la posibilidad de que el contrato reciba Ether adicional entre líneas, a menos que se restrinja explícitamente. Por eso, no concluye necesariamente que la tercera transferencia fallará, aunque el balance inicial esté bien definido.

Finalmente, la función no incluye una postcondición que afirme que la ejecución debe completarse sin fallos. En ausencia de tal especificación, el SMTChecker considera que una ruta de ejecución que termina en `revert` es válida, siempre que no contradiga una propiedad activa.

En cambio, nuestra herramienta rastrea el balance a lo largo de cada instrucción, modelando el consumo de fondos en cada transferencia. Al detectar que las dos primeras transferencias agotan los recursos disponibles, infiere correctamente que la tercera transferencia está destinada a fallar. Por ello, informa un `revert` inevitable, incluso sin necesidad de una aserción final.

5.2. Slither

Ejemplo simple

A continuación, ejecutamos el ejemplo 4.1 con la herramienta Slither.

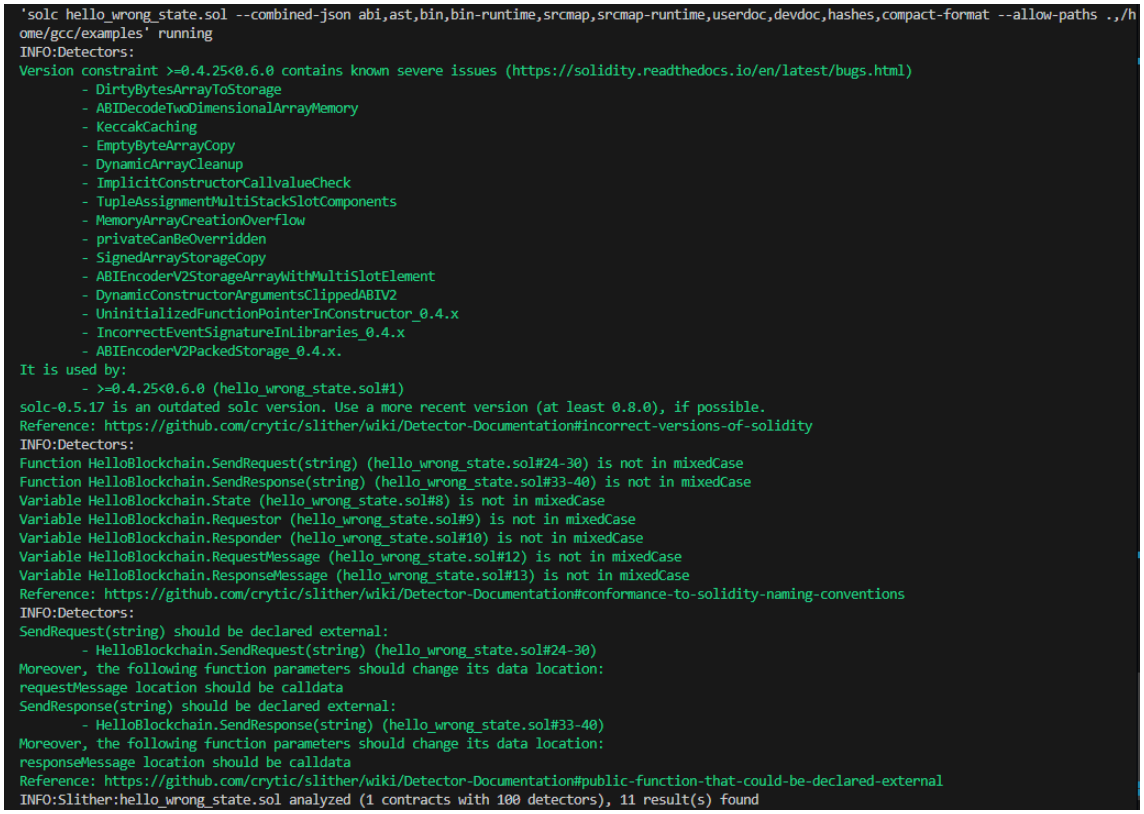


Fig. 5.1: Salida de Slither para el ejemplo simple

Como se puede observar, Slither detecta varios problemas, tales como vulnerabilidades relacionadas con la versión del compilador de Solidity, convenciones incorrectas en la nomenclatura de variables, y funciones que podrían ser declaradas como `external`. Sin embargo, este ejemplo también muestra que Slither no da prioridad a la verificación del estado correcto de una variable de estado, aun cuando existe una instrucción `assert` explícita.

Ejemplo del Bazaar

A continuación, aplicamos Slither al ejemplo del contrato `Bazaar`:

```
'solc --version' running
'solc bazaar_reverts.sol --combined-json abi,ast,bin,bin-runtime,srcmap,srcmap-runtime,userdoc,devdoc,hashes --allow-paths .,/home/gcc/examples'
running
INFO:Detectors:
Bazaar.ValidPrice(int256) (bazaar_reverts.sol#11-16) compares to a boolean constant:
        -assert(bool)(result == true) (bazaar_reverts.sol#14)
ItemListing.BuyItemNoRevert() (bazaar_reverts.sol#29-36) compares to a boolean constant:
        -assert(bool)(flag == true) (bazaar_reverts.sol#35)
ItemListing.BuyItemRevert_ReturnsTrue() (bazaar_reverts.sol#39-46) compares to a boolean constant:
        -assert(bool)(flag == true) (bazaar_reverts.sol#45)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#boolean-equality
INFO:Detectors:
Version constraint ^0.8.0 contains known severe issues (https://solidity.readthedocs.io/en/latest/bugs.html)
        - FullInlinerNonExpressionSplitArgumentEvaluationOrder
        - MissingSideEffectsOnSelectorAccess
        - AbiReencodingHeadOverflowWithStaticArrayCleanup
        - DirtyBytesArrayToStorage
        - DataLocationChangeInInternalOverride
        - NestedCalldataArrayAbiReencodingSizeValidation
        - SignedImmutables
        - ABIDecodeTwoDimensionalArrayMemory
        - KeccakCaching.
It is used by:
        - ^0.8.0 (bazaar_reverts.sol#2)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity
INFO:Detectors:
Function Bazaar.ValidPrice(int256) (bazaar_reverts.sol#11-16) is not in mixedCase
Variable Bazaar.InstanceBazaarMaintainer (bazaar_reverts.sol#5) is not in mixedCase
Function ItemListing.BuyItemRevert_WrongCall() (bazaar_reverts.sol#23-26) is not in mixedCase
Function ItemListing.BuyItemNoRevert() (bazaar_reverts.sol#29-36) is not in mixedCase
Function ItemListing.BuyItemRevert_ReturnsTrue() (bazaar_reverts.sol#39-46) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
INFO:Detectors:
Bazaar.InstanceBazaarMaintainer (bazaar_reverts.sol#5) should be immutable
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#state-variables-that-could-be-declared-immutable
INFO:Slither:bazaar_reverts.sol analyzed (2 contracts with 100 detectors), 10 result(s) found
```

Fig. 5.2: Salida de Slither para el ejemplo del Bazaar

En este caso, Slither no logra identificar la violación de la precondición ni la lógica incorrecta de la función que retorna `true` y ejecuta un `revert`. Esto resulta consistente con el comportamiento observado en el ejemplo anterior, y posiciona a Slither en un nivel de análisis comparable al del SMTChecker, al menos respecto a estas clases de errores.

Ejemplo de Bet

Finalmente, evaluamos el contrato de apuestas:

```
'solc bet_call_no_founds.sol --combined-json abi,ast,bin,bin-runtime,srcmap,srcmap-runtime,userdoc,devdoc,hashes --allow-paths .,/home/gcc/exampl
es' running
INFO:Detectors:
Bet.timeout() (bet_call_no_founds.sol#40-47) uses a dangerous strict equality:
        - require(bool,string)(address(this).balance == 2 * wager,Invalid Balance) (bet_call_no_founds.sol#42)
Bet.win(uint256) (bet_call_no_founds.sol#30-38) uses a dangerous strict equality:
        - require(bool,string)(address(this).balance == 2 * wager,Invalid Balance) (bet_call_no_founds.sol#33)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#dangerous-strict-equalities
INFO:Detectors:
Bet.constructor(address,address,uint256)._oracle (bet_call_no_founds.sol#13) lacks a zero-check on :
                - oracle = _oracle (bet_call_no_founds.sol#15)
Bet.constructor(address,address,uint256)._player2 (bet_call_no_founds.sol#13) lacks a zero-check on :
                - player2 = _player2 (bet_call_no_founds.sol#17)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#missing-zero-address-validation
INFO:Detectors:
Version constraint ^0.8.0 contains known severe issues (https://solidity.readthedocs.io/en/latest/bugs.html)
        - FullInlinerNonExpressionSplitArgumentEvaluationOrder
        - MissingSideEffectsOnSelectorAccess
        - AbiReencodingHeadOverflowWithStaticArrayCleanup
        - DirtyBytesArrayToStorage
        - DataLocationChangeInInternalOverride
        - NestedCalldataArrayAbiReencodingSizeValidation
        - SignedImmutables
        - ABIDecodeTwoDimensionalArrayMemory
        - KeccakCaching.
It is used by:
        - ^0.8.0 (bet_call_no_founds.sol#2)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#incorrect-versions-of-solidity
INFO:Detectors:
Low level call in Bet.win(uint256) (bet_call_no_founds.sol#30-38):
        - (success,None) = addressWinner.call{value: address(this).balance}() (bet_call_no_founds.sol#35)
Low level call in Bet.timeout() (bet_call_no_founds.sol#40-47):
        - (success,None) = player1.call{value: wager}() (bet_call_no_founds.sol#43)
        - (success2,None) = player2.call{value: wager}() (bet_call_no_founds.sol#45)
Low level call in Bet.timeout_wrong() (bet_call_no_founds.sol#49-59):
        - (success,None) = player1.call{value: wager}() (bet_call_no_founds.sol#52)
        - (success2,None) = player2.call{value: wager}() (bet_call_no_founds.sol#54)
        - (success3,None) = player2.call{value: wager}() (bet_call_no_founds.sol#57)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#low-level-calls
INFO:Detectors:
Function Bet.timeout_wrong() (bet_call_no_founds.sol#49-59) is not in mixedCase
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#conformance-to-solidity-naming-conventions
INFO:Detectors:
Bet.deadline (bet_call_no_founds.sol#8) should be immutable
Bet.oracle (bet_call_no_founds.sol#9) should be immutable
Bet.player1 (bet_call_no_founds.sol#6) should be immutable
Bet.player2 (bet_call_no_founds.sol#7) should be immutable
Bet.wager (bet_call_no_founds.sol#10) should be immutable
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#state-variables-that-could-be-declared-immutable
INFO:Slither:bet_call_no_founds.sol analyzed (1 contracts with 100 detectors), 14 result(s) found
```

Fig. 5.3: Salida de Slither para el ejemplo de apuesta

En este caso, Slither emite algunas advertencias sobre la condición `address(this).balance == 2 * wager`, sugiriendo que podría ser más apropiado utilizar una comparación con `>=`, ya que un balance mayor también sería válido. No obstante, no logra reconocer que la tercera transferencia inevitablemente fallará por falta de fondos, a pesar de haber detectado la validación del balance al inicio de la función. Este resultado refuerza la limitación de Slither para realizar un análisis profundo de las implicancias dinámicas del control de flujo y el consumo de recursos.

5.3. Tabla comparativa

A continuación, presentamos una tabla detallada que resume los resultados comparativos de las herramientas evaluadas, utilizando un workbench desarrollado específicamente para este trabajo. Este workbench se compone de contratos modificados con errores introducidos manualmente, basados en ejemplos de Rosetta y Azure.

	Detects invalid state variable			*Inter-contract communication*			*Call without enough funds*		
	SMT	**STLH**	**Slither**	**SMT**	**STLH**	**Slither**	**SMT**	**STLH**	**Slither**
Hello-wrong-state	✓	✓	×						
Bazaar-reverts				×	✓	×			
Bet-no-funds							×	✓	×
BasicProvenance-1	✓	✓	×						
BasicProvenance-2	✓	✓	×						
DigitalLocker-1	✓	✓	×						
RoomThermostat-1	✓	✓	×						
SimpleMarketplace-1	✓	✓	×						
SimpleTransfer-1							×	✓	×

Tab. 5.1: Tool comparison results

6. CONCLUSIÓN

En este trabajo se presentó una herramienta novedosa de análisis estático para contratos inteligentes escritos en Solidity, demostrando su capacidad para identificar rutas de fallo sutiles que suelen pasar desapercibidas para el verificador integrado SMTChecker. Nuestra aproximación se diferencia en dos aspectos fundamentales: (1) modela de manera más precisa las llamadas externas y el balance del contrato, rastreando las transiciones de estado a lo largo del flujo de control; y (2) clasifica cualquier ejecución que conduzca inevitablemente a un `revert` o a código inalcanzable como una falla potencial, en lugar de asumir que los `revert` son resultados válidos por defecto.

A través de una serie de ejemplos representativos, mostramos cómo la herramienta identifica condiciones que incluyen entradas inválidas (como precios negativos), transferencias que exceden el balance disponible y caminos lógicos que no pueden ser alcanzados. Muchos de estos escenarios no son reportados como errores por análisis estándar.

Si bien este enfoque mejora la detección de ciertos modos de fallo, implica también algunas compensaciones. Un modelo más preciso puede conllevar un mayor costo computacional y requerir conocimientos específicos del dominio, como restricciones de gas o interacciones entre contratos. Observamos que, aunque el SMTChecker aplica supuestos conservadores—como modelar el código externo como desconocido—, no trata automáticamente cada `revert` como una violación de la correctitud. Esto subraya la importancia de especificar claramente las propiedades deseadas de correctitud y los posibles escenarios de fallo desde la etapa de diseño.

6.1. Trabajo futuro

Este proyecto sienta las bases para el desarrollo de un traductor y verificador estático robusto que conecte Solidity con Haskell enriquecido con tipos refinados. Algunas líneas de trabajo futuras incluyen:

- Realizar una prueba de correctitud de la herramienta.
- Incorporar soporte para estructuras de datos como arreglos (`arrays`) y diccionarios (`mappings`).
- Extender la verificación para cubrir propiedades aritméticas, como desbordamientos y divisiones inválidas.
- Analizar y eventualmente soportar eventos (`events`) en contratos inteligentes.
- Mejorar la arquitectura y eficiencia del pipeline de traducción y verificación.
- Agregar soporte para interfaces y contratos abstractos.

BIBLIOGRAFíA

[1] B. Tan et al. "SolType: Refinement Types for Arithmetic Overflow in Solidity". En: *Proc. ACM Program. Lang.* 6.POPL (2022).

[2] N. Ivanov et al. "Security Threat Mitigation for Smart Contracts: A Comprehensive Survey". En: *ACM Computing Surveys* (2023).

[3] M. di Angelo. "A Survey of Tools for Analyzing Ethereum Smart Contracts". En: (2024).

[4] Azure Samples. *Azure Blockchain Workbench: Application and Smart Contract Samples*. https://github.com/Azure-Samples/blockchain/tree/master/blockchain-workbench/application-and-smart-contract-samples. [Online; accessed 29-January-2025]. 2025.

[5] Sidi Mohamed Beillahi et al. "Automated auditing of price gouging TOD vulnerabilities in smart contracts". En: *2022 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, mayo de 2022.

[6] Blockchain UNICA. *Rosetta Smart Contracts*. https://github.com/blockchain-unica/rosetta-smart-contracts. [Online; accessed 29-January-2025]. 2025.

[7] Lorenz Breidenbach et al. "An in-depth look at the parity multisig bug". En: *Hack., Distrib.* (2017).

[8] R. Browne. *Accidental Bug May Have Frozen $280 Million Worth of Digital Coin Ether in a Cryptocurrency Wallet*. Retrieved from https://www.cnbc.com/2017/11/08/accidental-bug-may-have-frozen-280-worth-of-etheron-parity-wallet.html. 2017.

[9] Josselin Feist, Gustavo Grieco y Alex Groce. "Slither: A static analysis framework for smart contracts". En: *arXiv [cs.SE]* (ago. de 2019).

[10] Ernesto Frontera. *A History of the DAO Hack*. Retrieved from https://coinmarketcap.com/alexandria/article/ahistory-of-the-dao-hack. 2022.

[11] *Glasgow Haskell Compiler — The Glasgow Haskell Compiler*. en. https://www.haskell.org/ghc/. Accessed: 2025-2-18.

[12] Giacomo Ibba et al. "A curated solidity smart contracts repository of metrics and vulnerability". En: *Proceedings of the 20th International Conference on Predictive Models and Data Analytics in Software Engineering*. New York, NY, USA: ACM, jul. de 2024.

[13] PMD. *An Extensible Cross-language Static Code Analyzer*. Accessed: Feb. 18, 2025. [Online]. Available: https://pmd.github.io/. 2025.

[14] *Refinement Types*. en. https://ucsd-progsys.github.io/liquidhaskell-tutorial/Tutorial_03_Basic.html. Accessed: 2025-2-7.

[15] Solidity Developers. *Solidity SMT Checker Documentation*. https://docs.soliditylang.org/en/latest/smtchecker.html. [Online; accessed 29-January-2025]. 2025.

[16] Solidity Parser Contributors. *solidity-parser: A Solidity AST Parser for JavaScript*. https://github.com/solidity-parser/parser. [Online; accessed 29-January-2025]. 2025.

[17] SonarQube. *Code Quality and Code Security*. Accessed: Feb. 18, 2025. [Online]. Available: https://www.sonarqube.org/. 2025.

[18] SpotBugs. *Find Bugs in Java Programs*. Accessed: Feb. 18, 2025. [Online]. Available: https://spotbugs.github.io/. 2025.

[19] Ştefan-Claudiu Susan y Andrei Arusoaie. "Identifying vulnerabilities in smart contracts using interval analysis". en. En: *Electron. Proc. Theor. Comput. Sci.* 389 (sep. de 2023), págs. 144-151.

[20] Niki Vazou et al. "Refinement types for Haskell". En: *Proceedings of the 19th ACM SIGPLAN international conference on Functional programming*. New York, NY, USA: ACM, ago. de 2014.

[21] Haijun Wang et al. "Skyeye: Detecting imminent attacks via analyzing adversarial smart contracts". En: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. New York, NY, USA: ACM, oct. de 2024, págs. 1570-1582.

[22] William Zhang et al. "MPro: Combining static and symbolic analysis for scalable testing of smart contract". En: *arXiv [cs.CR]* (nov. de 2019).

7. APPENDIX

```
{-@ LIQUID "--exact-data-cons" @-}
{-@ LIQUID "--ple"      @-}
{-@ LIQUID "--no-termination"        @-}

module Bazaar where

data Message = Message {
  sender :: Address,
  value :: Integer
} deriving (Show, Eq)

{-@ data Message = Message {
  sender :: Address,
  value :: Integer}
@-}

data Block = Block {
  number :: Integer}

{-@ data Block = Block {
  number :: Integer}
@-}

{-@ type Amount = {v: Integer | v >= 0} @-}
type Amount = Integer

type Address = Integer
{-@ type Address = Integer @-}

{-@ revert :: {v:String | False} -> a  @-}
revert :: String -> a
revert  = error


{-@ callBazaar :: ctx:BazaarState
           -> Address
           -> amount: {Amount | 0 <= amount && amount <= balanceBazaar ctx}
           -> ctx':{ BazaarState | balanceBazaar ctx' == balanceBazaar ctx
           - amount  && (msgBazaar ctx' == msgBazaar ctx) && (blockBazaar
            ctx' == blockBazaar ctx) && (instanceBazaarMaintainerBazaar
            ctx' == instanceBazaarMaintainerBazaar ctx)}
@-}
callBazaar:: BazaarState -> Address -> Amount -> BazaarState
callBazaar ctx addr amount = ctx { balanceBazaar =
                                          balanceBazaar ctx - amount }


{-@ data BazaarState = BazaarState {
  msgBazaar :: Message,
  blockBazaar :: Block,
  balanceBazaar :: Amount,
  instanceBazaarMaintainerBazaar :: Address
}
```

```
54 @-}
55
56 data BazaarState = BazaarState {
57   msgBazaar :: Message,
58   blockBazaar :: Block,
59   balanceBazaar :: Amount,
60   instanceBazaarMaintainerBazaar :: Address
61 }
62
63 {-@ constructorBazaar :: Message -> Block  -> {v:BazaarState | True} @-}
64 constructorBazaar :: Message -> Block -> BazaarState
65 constructorBazaar msgctx block  = BazaarState
66             { msgBazaar = msgctx,
67               blockBazaar = block,
68               balanceBazaar = 0,
69               instanceBazaarMaintainerBazaar = sender msgctx
70             }
71
72 {-@ validPrice :: {st:BazaarState | True} -> {itemPrice:Integer |
73             ((itemPrice) > (0))} -> {v:Bool | ((v) == (True))} @-}
74 validPrice :: BazaarState -> Integer -> Bool
75 validPrice st itemPrice =
76   let result = True in
77   result
78
79
80
81 {-@ callItemListing :: ctx:ItemListingState
82             -> Address
83             -> amount: {Amount | 0 <= amount && amount <= balanceItemListing
                  ctx}
84             -> ctx':{ ItemListingState | balanceItemListing ctx' ==
85             balanceItemListing ctx - amount  && (msgItemListing ctx' ==
86             msgItemListing ctx) && (blockItemListing ctx' ==
                  blockItemListing
87             ctx) && (flagItemListing ctx' == flagItemListing ctx)}
88 @-}
89 callItemListing:: ItemListingState -> Address -> Amount -> ItemListingState
90 callItemListing ctx addr amount = ctx { balanceItemListing =
91                                         balanceItemListing ctx - amount }
92
93
94 {-@ data ItemListingState = ItemListingState {
95   msgItemListing :: Message,
96   blockItemListing :: Block,
97   balanceItemListing :: Amount,
98   flagItemListing :: Bool
99 }
100 @-}
101
102 data ItemListingState = ItemListingState {
103   msgItemListing :: Message,
104   blockItemListing :: Block,
105   balanceItemListing :: Amount,
106   flagItemListing :: Bool
107 }
108
109 {-@ buyItemRevert_WrongCall :: {st:ItemListingState | True} ->
```

```
                                    {v:ItemListingState | True} @-}
buyItemRevert_WrongCall :: ItemListingState -> ItemListingState
buyItemRevert_WrongCall st = let
  bazaar = constructorBazaar (msgItemListing st) (blockItemListing st)
  st1 = st { flagItemListing = validPrice bazaar (-(5)) }
    in
      st1

{-@ buyItemNoRevert :: {st:ItemListingState | True} -> {v:ItemListingState
    |
                                    (((flagItemListing v)) == (True))} @-}
buyItemNoRevert :: ItemListingState -> ItemListingState
buyItemNoRevert st = let
  bazaar = constructorBazaar (msgItemListing st) (blockItemListing st)
  st1 = if (not(validPrice bazaar 50))
    then
    let
      st1 = revert "It is not a valid price"
        in
          st1
    else st
  st2 = st1 { flagItemListing = True }
  st3 = st2 -- assert statement
    in
      st3

{-@ buyItemRevert_ReturnsTrue :: {st:ItemListingState | True} ->
        {v:ItemListingState | (((flagItemListing v)) == (True))} @-}
buyItemRevert_ReturnsTrue :: ItemListingState -> ItemListingState
buyItemRevert_ReturnsTrue st = let
  bazaar = constructorBazaar (msgItemListing st) (blockItemListing st)
  st1 = if validPrice bazaar 50
    then
    let
      st1 = revert "It is not a valid price"
        in
          st1
    else st
  st2 = st1 { flagItemListing = True }
  st3 = st2 -- assert statement
    in
      st3
```

Listing 7.1: Translation of 4.1

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.0;

contract Bet {

    address payable player1;
    address payable player2;
    uint deadline;
    address oracle;
    uint wager;
    bool open;

    constructor(address _oracle, address payable _player2, uint _timeout)
        payable {
```

```
14          wager = msg.value;
15          oracle = _oracle;
16          player1 = payable(msg.sender);
17          player2 = _player2;
18          deadline = block.number + _timeout;
19          open = true;
20      }
21
22      function join() payable public {
23          require(msg.value == wager, "invalid value");
24          require(player2 == msg.sender, "invalid player");
25          require(open, "bets are closed");
26          require(deadline >= block.number, "time out");
27          open = false;
28      }
29
30      function win(uint winner) external  {
31          require(msg.sender == oracle, "only the oracle");
32          require(winner <=1, "Invalid winner selector");
33          require(address(this).balance == 2*wager, "Invalid Balance");
34          address payable addressWinner = (winner == 0) ? player1 : player2;
35          (bool success,) = addressWinner.call{value: address(this).balance}(
                """);
36          require (success, "Transfer failed.");
37
38      }
39
40      function timeout() external payable {
41          require(deadline < block.number, "The bets are still open");
42          require(address(this).balance == 2*wager, "Invalid Balance");
43          (bool success,) = player1.call{value: wager}("");
44          require (success, "Transfer failed.");
45          (bool success2,) = player2.call{value: wager}("");
46          require (success2, "Transfer failed.");
47      }
48  }
```

Listing 7.2: Bet contract from Rosetta

```
1  {-@ LIQUID "--exact-data-cons" @-}
2  {-@ LIQUID "--ple"       @-}
3  {-@ LIQUID "--no-termination"        @-}
4
5  module Bet where
6
7  data Message = Message {
8    sender :: Address,
9    value :: Integer
10 } deriving (Show, Eq)
11
12 {-@ data Message = Message {
13   sender :: Address,
14   value :: Integer}
15 @-}
16
17 data Block = Block {
18   number :: Integer}
19
20 {-@ data Block = Block {
```

```
21    number :: Integer}
22  @-}
23
24  {-@ type Amount = {v: Integer | v >= 0} @-}
25  type Amount = Integer
26
27  type Address = Integer
28  {-@ type Address = Integer @-}
29
30  {-@ revert :: {v:String | False} -> a  @-}
31  revert :: String -> a
32  revert  = error
33
34
35  {-@ callBet :: ctx:BetState
36              -> Address
37              -> amount: {Amount | 0 <= amount && amount <= balanceBet ctx}
38              -> ctx':{ BetState | balanceBet ctx' == balanceBet ctx - amount
39                       && (msgBet ctx' == msgBet ctx)
40                       && (blockBet ctx' == blockBet ctx)
41                       && (player1Bet ctx' == player1Bet ctx)
42                       && (player2Bet ctx' == player2Bet ctx)
43                       && (deadlineBet ctx' == deadlineBet ctx)
44                       && (oracleBet ctx' == oracleBet ctx)
45                       && (wagerBet ctx' == wagerBet ctx)
46                       && (openBet ctx' == openBet ctx)}
47  @-}
48  callBet:: BetState -> Address -> Amount -> BetState
49  callBet ctx addr amount = ctx { balanceBet = balanceBet ctx - amount }
50
51
52  {-@ data BetState = BetState {
53    msgBet :: Message,
54    blockBet :: Block,
55    balanceBet :: Amount,
56    player1Bet :: Address,
57    player2Bet :: Address,
58    deadlineBet :: Integer,
59    oracleBet :: Address,
60    wagerBet :: Integer,
61    openBet :: Bool
62  }
63  @-}
64
65  data BetState = BetState {
66    msgBet :: Message,
67    blockBet :: Block,
68    balanceBet :: Amount,
69    player1Bet :: Address,
70    player2Bet :: Address,
71    deadlineBet :: Integer,
72    oracleBet :: Address,
73    wagerBet :: Integer,
74    openBet :: Bool
75  }
76
77  {-@ constructorBet :: Message -> Block  -> {_oracle:Address | True} ->
78                          {_player2:Address | True} ->
```

```
79                          {_timeout:Integer | True} ->
80                          {v:BetState | True} @-}
81  constructorBet :: Message -> Block -> Address -> Address -> Integer ->
        BetState
82  constructorBet msgctx block _oracle _player2 _timeout = BetState
83              { msgBet = msgctx,
84                blockBet = block,
85                balanceBet = 0,
86                oracleBet = _oracle,
87                player1Bet = (sender msgctx),
88                player2Bet = _player2,
89                deadlineBet = ((number block) + _timeout),
90                openBet = True
91              }
92
93  {-@ join :: {st:BetState | (((value (msgBet st))) == ((wagerBet st))) &&
94                          (((player2Bet st)) == ((sender (msgBet st))))
95                          && (openBet st)
96                          && (((deadlineBet st)) >= ((number (blockBet st))))
97                          } -> {v:BetState | True}
98  @-}
99  join :: BetState -> BetState
100 join st = let
101   st1 = if ((value (msgBet st)) == (wagerBet st)) then st else revert "
          error"
102   st2 = if ((player2Bet st1) == (sender (msgBet st1))) then st1 else revert
           "error"
103   st3 = if (openBet st2) then st2 else revert "error"
104   st4 = if ((deadlineBet st3) >= (number (blockBet st3)))
105               then st3 else revert "error"
106   st5 = st4 { openBet = False }
107     in
108       st5
109
110 {-@ win :: {st:BetState | (((sender (msgBet st))) == ((oracleBet st)))
111                          && (((balanceBet st)) == ((2 * (wagerBet st))))}
112                          -> {winner:Integer | ((winner) <= (1))}
113                          -> {v:BetState | True}
114 @-}
115 win :: BetState -> Integer -> BetState
116 win st winner = let
117   st1 = if ((sender (msgBet st)) == (oracleBet st)) then st else revert "
          error"
118   st2 = if (winner <= 1) then st1 else revert "error"
119   st3 = if ((balanceBet st2) == (2 * (wagerBet st2))) then st2 else revert
          "error"
120   addressWinner = if (winner == 0) then (player1Bet st) else (player2Bet st
          )
121   st4 = callBet st3 (addressWinner) (balanceBet st3)
122   st5 = st4 -- skipped require statement
123     in
124       st5
125
126 {-@ timeout :: {st:BetState | (((deadlineBet st)) < ((number (blockBet st))
        ))
127                             && (((balanceBet st)) == ((2 * (wagerBet st))))
                                }
128                             -> {v:BetState | True} @-}
```

```
129 timeout :: BetState -> BetState
130 timeout st = let
131   st1 = if ((deadlineBet st) < (number (blockBet st))) then st else revert
        "error"
132   st2 = if ((balanceBet st1) == (2 * (wagerBet st1))) then st1 else revert
        "error"
133   st3 = callBet st2 ((player1Bet st2)) ((wagerBet st2))
134   st4 = st3 -- skipped require statement
135   st5 = callBet st4 ((player2Bet st4)) ((wagerBet st4))
136   st6 = st5 -- skipped require statement
137     in
138       st6
```

Listing 7.3: Translation of 7.2